

Pilha Tecnológica — STACK

Template oficial da disciplina Projeto – Inovações Tecnológicas (Feevale)

Instruções: preencha todas as seções abaixo, commite este arquivo como STACK.docx na raiz do repositório GitHub e mantenha-o atualizado ao longo da Etapa 3. Tecnologias incorporadas ao projeto sem estarem registradas neste documento não serão consideradas na avaliação.

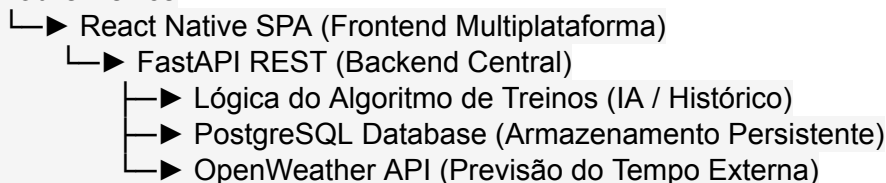
S1 — Identificação

- **Nome do produto:** RunAI
- **Integrantes:** Gabriel Dilly (Backend) e Vitor Zanella (Frontend)
- **Repositório GitHub:** <https://github.com/RunAI-project/runai-app>
- **Data de criação:** 25/09/2026
- **Versão do documento:** 1.0

S2 — Arquitetura Geral

O sistema do RunAI segue uma arquitetura cliente-servidor focada em aplicativos móveis. O Frontend foi planejado como um aplicativo mobile multiplataforma, desenvolvido em React Native, alinhando aos atributos de produto listados na seção "É/Não É". O Backend será consumido como uma API RESTful, programado em Python com FastAPI, onde a lógica principal do algoritmo de geração dinâmica e individualizada de treinos. Os dados (perfil do usuário, histórico de exercícios, adaptações por dores ou falta de descanso, preferências de dias na semana) serão armazenados de forma estruturada em um Banco de Dados relacional, o PostgreSQL. Adicionalmente, o sistema integrará consumo de APIs externas (como OpenWeather) para auxiliar na funcionalidade climática exigida no MVP.

Mobile Device



S3 — Tabela de Tecnologias

Tecnologia	Versão	Camada	Finalidade	Justificativa
React Native	0.74.x	Frontend	Construção das interfaces do App Mobile	Garante desenvolvimento multiplataforma (requisito fundamental mapeado no Canvas É/Não É) para engajar corredores de ambas as plataformas com uma base de código unificada.
Python (FastAPI)	3.12	Backend	Servidor da API, lógica de negócio e cálculos de plano de treino	O Python oferece um ecossistema excelente para trabalhar com algoritmos de recomendação e processamento de dados para gerar os treinos baseados no desempenho. FastAPI traz agilidade e alta performance no tráfego de dados.

Tecnologia	Versão	Camada	Finalidade	Justificativa
PostgreSQL	16	Banco de Dados	Persistência de histórico, perfil físico e progresso dos corredores	Garante a integridade relacional entre treinos executados, dias de descanso e evolução pessoal, essencial para a adaptação e consistência do usuário (Insight 3).
OpenWeather API	V2.5	Serviço Externo	Obtenção do estado climático e previsão de horários	Integração necessária para viabilizar a feature MVP (F18 - Consultar a previsão do tempo para auxiliar na escolha do horário).

S4 — Ferramentas de Desenvolvimento

- **IDE(s):** VS Code, Cursor
- **Gerenciador de pacotes:** npm ou yarn (Frontend), pip ou poetry (Backend)
- **Linter / formatter:** ESLint + Prettier (React Native), Black + Flake8 (Python)
- **Testes:** Jest (Frontend), Pytest (Backend)
- **CI/CD:** GitHub Actions

S5 — Declaração de Uso de Inteligência Artificial

Requisito obrigatório de integridade acadêmica.

[X] A equipe UTILIZA Inteligência Artificial

Ferramenta(s): Gemini, ChatGPT

Finalidade(s): Complementar e auxiliar no desenvolvimento do projeto. Auxílio em busca de informações sobre o assunto do projeto, tanto para conhecimento ou desenvolvimento.

Protocolo de revisão: Todo o código será solicitado com o melhor detalhamento possível da solicitação e será lido pelos participantes do grupo para entendimento.

[] A equipe NÃO utiliza Inteligência Artificial

Declaração: "Nenhuma ferramenta de Inteligência Artificial será utilizada neste projeto."

Este documento deve ser commitado na raiz do repositório como STACK.docx e mantido atualizado ao longo da Etapa 3. Alterações de pilha devem ser registradas com data e justificativa.